Notice the empty element at the end of the array? That's caused by the full-stop character at the end of the string, and empty matches can be suppressed by adding a PREG_* constant as a flags: parameter to the call: preg_split("/[^a-zA-Z]+/", $text, flags: PREG_SPLIT_NO_EMPTY).

preg_last_error(), preg_last_error_message()

Finally, we have the two error-handling preg_* functions. preg_last_error() returns an integer value indicating the error status of the last PCRE function call made, which is mapped by the following PCRE constants:

Table 8-36. *PREG constants*

PREG_NO_ERROR	No errors
PREG_INTERNAL_ERROR	PCRE error state not covered by any of the other conditions
PREG_BACKTRACK_LIMIT_ERROR	Backtrack limit exceeded
PREG_RECURSION_LIMIT_ERROR	Recursion limit exceeded
PREG_BAD_UTF8_ERROR	Malformed UTF-8 detected (when running in UTF-8 mode)
PREG_BAD_UTF8_OFFSET_ERROR	Function is called with an offset resulting in malformed UTF-8
PREG_JIT_STACKLIMIT_ERROR	If JIT pattern compilation is enabled and the stack size exceeded

JIT compilation, backtracking, and recursion limits are all configurable via the usual means. I encourage you to consult the official PHP documentation for the full list of PCRE constants and configuration parameters. The typical use case for preg_last_error() is to test for PREG_NO_ERROR and handle accordingly. The following example is adapted from the PHP official documentation:

```
$text = 'foobar foobar foobar';
preg_match('/(?:\D+|<\d+>)*[!?]/', $text);
if (preg_last_error() !== PREG_NO_ERROR) {
    print preg_last_error_msg();
}
```

```
Output:
Backtrack limit exhausted
```